

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 2: ANALYSIS OF ALGORITHMS

ABDULLAH BAL, PHD

Objectives



Recap
Data Structures



Analysis of
Algorithms



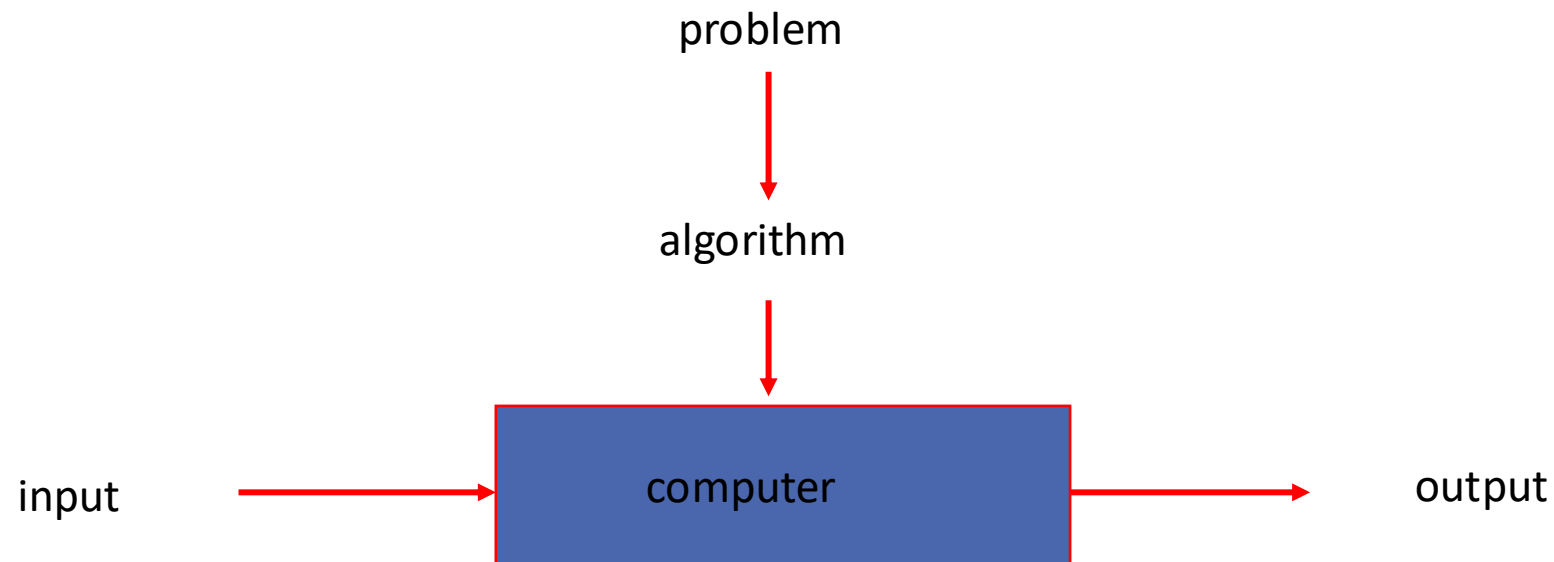
Euclid's, Sieve,
Sequential
Search



Order of
Growth
Comparison

What is an algorithm?

- An algorithm is any well-defined **computational procedure** that takes some value, or set of values, as **input** and produces some value, or set of values, as **output** in a **finite amount of time**.
- An algorithm as **a tool** for solving a well-specified computational problem.

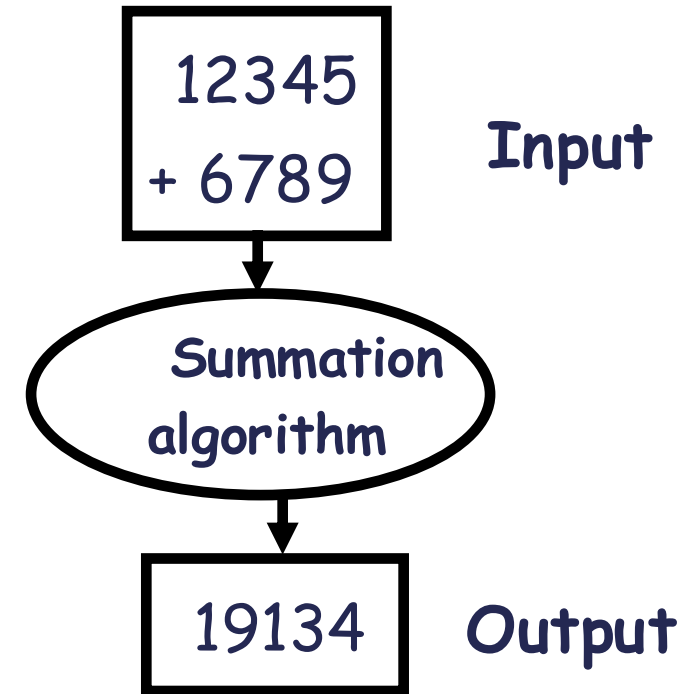


What is an algorithm?

Function: Input $\rightarrow$ Output

Definition:

Step-by-step mechanical procedure that computes a function for any input



Computing $\gcd(m,n)$?

- Compute $\gcd(60,24)$?
- The method is simply based on the definition of **the greatest common divisor** of m and n as the largest integer that divides both numbers evenly.
- Consecutive integer checking algorithm
 - Step 1** Assign the value of $\min\{m,n\}$ to t
 - Step 2** Divide m by t . If the remainder is 0, go to Step 3; otherwise, go to Step 4
 - Step 3** Divide n by t . If the remainder is 0, return t and stop; otherwise, go to Step 4
 - Step 4** Decrease t by 1 and go to Step 2

Other methods for $\gcd(m,n)$ [cont.]

- Middle-school procedure

- **Step 1** Find the prime factorization of m
- **Step 2** Find the prime factorization of n
- **Step 3** Find all the common prime factors
- **Step 4** Compute the product of all the common prime factors and return it as $\gcd(m,n)$

- Thus, for the numbers 60 and 24, we get

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\gcd(60, 24) = 2 \cdot 2 \cdot 3 = 12.$$

- $\gcd(571\,787, 912\,673)=?$

Sieve of Eratosthenes

- The algorithm starts by initializing a list of prime candidates with consecutive integers from 2 to n .
- Then, on its first iteration, the algorithm eliminates from the list all multiples of 2, i.e., 4, 6, and so on.
- Then it moves to the next item on the list, which is 3, and eliminates its multiples.
- No pass for number 4 is needed: since 4 itself and all its multiples are also multiples of 2, they were already eliminated on a previous pass.
- The next remaining number on the list, which is used on the third pass, is 5.
- The algorithm continues in this fashion until no more numbers can be eliminated from the list. The remaining integers of the list are the primes needed.

As an example, consider the application of the algorithm to finding the list of primes not exceeding $n = 25$:

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3		5		7		9		11		13		15		17		19		21		23		25
2	3		5		7				11		13				17		19				23		25
2	3		5		7				11		13				17		19				23		

Sieve of Eratosthenes

ALGORITHM *Sieve*(n)

//Implements the sieve of Eratosthenes

//Input: A positive integer $n > 1$

//Output: Array L of all prime numbers less than or equal to n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do** //see note before pseudocode

if $A[p] \neq 0$ // p hasn't been eliminated on previous passes

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ //mark element as eliminated

$j \leftarrow j + p$

//copy the remaining elements of A to array L of the primes

$i \leftarrow 0$

for $p \leftarrow 2$ **to** n **do**

if $A[p] \neq 0$

$L[i] \leftarrow A[p]$

$i \leftarrow i + 1$

return L

Example: Euclid's Algorithm

- Problem: Find $\gcd(m, n)$, the greatest common divisor of two nonnegative, not both zero integers m and n
- Examples: $\gcd(60, 24) = 12$, $\gcd(60, 0) = 60$, $\gcd(0, 0) = ?$
- Euclid's algorithm is based on repeated application of equality

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

until the second number becomes 0, which makes the problem trivial.

- Example: $\gcd(60, 24) = \gcd(24, 12) = \gcd(12, 0) = 12$

Two descriptions of Euclid's algorithm

Step 1 If $n = 0$, return m and stop; otherwise go to Step 2

Step 2 Divide m by n and assign the value of the remainder to r

Step 3 Assign the value of n to m and the value of r to n . Go to Step 1.

Pseudocode:

```
while  $n \neq 0$  do
   $r \leftarrow m \bmod n$ 
   $m \leftarrow n$ 
   $n \leftarrow r$ 
return  $m$ 
```

Why study algorithms?

Theoretical importance

- The core of computer science

Practical importance

- A practitioner's **toolkit** of known algorithms
- Framework for designing and analyzing algorithms for new problems (**inspiration**)

Analysis of algorithms

How good is the algorithm?

- Time efficiency
- Space efficiency

Analyzing Algorithms

- ▶ Analyzing an algorithm has come to mean predicting the resources that the algorithm requires.
 - ▶ Memory
 - ▶ Communication bandwidth
 - ▶ Energy consumption
 - ▶ Computational time

Example: Insertion Sort

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

$A=[5,2,4,6,1,3]$

- **Output:** A permutation (reordering) of the input sequence.

$A=[1,2,3,4,5,6]$

Example: Insertion Sort

- It takes two parameters:
 - An array A containing the values to be sorted and the number n of values of sort.
 - The values occupy positions $A[1]$ through $A[n]$ of the array, which we denote by $A[1 : n]$.
 - When the INSERTION-SORT procedure is finished, array $A[1 : n]$ contains the original values, but in sorted order.

Example: Insertion Sort

- In each iteration, the blue rectangle holds the key taken from $A[i]$, which is compared with the values in tan rectangles to its left.
- Orange arrows show array values moved one position to the right, and blue arrows indicate where the key moves to.

ALGORITHM *InsertionSort*($A[0..n - 1]$)

//Sorts a given array by insertion sort

//Input: An array $A[0..n - 1]$ of n orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 1$ **to** $n - 1$ **do**

$v \leftarrow A[i]$

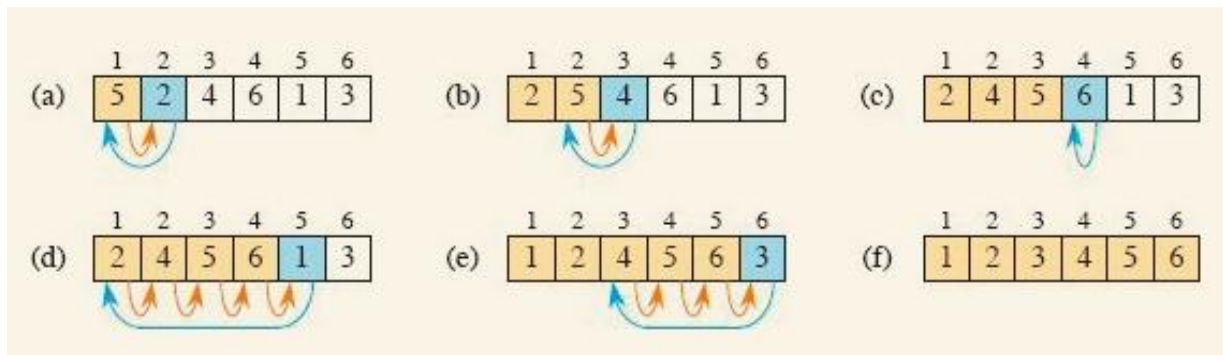
$j \leftarrow i - 1$

while $j \geq 0$ **and** $A[j] > v$ **do**

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j + 1] \leftarrow v$



Analysis of Insertion Sort

- How long does the INSERTION-SORT procedure take?
 - One way:

Run it on our computer and time how long it takes to run.

Analysis of Insertion Sort

- What would such a timing test tell us?
- We would find out how long insertion sort takes to run on
 - our particular computer,
 - on that particular input,
 - under the particular implementation that we created,
 - with the particular compiler or interpreter that we ran,
 - with the particular libraries that we linked in,
 - and with the particular background tasks that were running on our computer concurrently with our timing test (such as checking for incoming information over a network).

Analysis of Insertion Sort

- If we run insertion sort again on our computer with the same input, we might even get a **different timing result**.
- We need a way to predict, given a new input, how long insertion sort will take.
- Instead of timing a run, or even several runs, of insertion sort, we can determine how long it takes by **analyzing the algorithm itself**.

Analysis of Insertion Sort

- We'll examine how many times it executes each line of pseudocode and how long each line of pseudocode takes to run.
- We'll first come up with a precise but **complicated formula** for the running time.
- Then, we'll **distill the important part** of the formula using a convenient notation that can help us compare the running times of different algorithms for the same problem.

Analysis of Insertion Sort

- ▶ First, let's acknowledge that the **running time depends on the input**.
- ▶ It is not surprised that **sorting a thousand numbers** takes longer than **sorting three numbers**.
- ▶ Moreover, insertion sort can take different amounts of time to sort **two input arrays of the same size**, depending on how nearly sorted they already are.
- ▶ We need to define the terms “**running time**” and “**input size**” more carefully.
- ▶ We also need to be clear about whether we are discussing the running time for an input that elicits **the worst-case** behavior, **the best-case** behavior, or some other case.

Input Size

- For many problems, such as **sorting** or computing **discrete Fourier** transforms, the most natural measure is the number of items in the input—for example, the number n of items being sorted.
- For many other problems, such as multiplying two integers, the best measure of input size is the total **number of bits needed** to represent the input in ordinary binary notation.
- For example, if the input to an algorithm is a **graph**, we usually characterize the input size by both the **number of vertices** and the **number of edges** in the graph.

Running Time

- The running time of an algorithm on a particular input is the number of **instructions** and **data accesses** **executed**.
- How we account for these costs should be **independent** of any **particular computer**, but within the framework of the RAM model.
- A constant amount of time is required to execute each line of our **pseudocode**.
- One line might take more or less time than another line, but we'll assume that each execution of the k^{th} line takes c_k time, where **c_k is a constant**.

Running Time : The Best Case

- The best case occurs when the array is already sorted.
- The running time is thus a linear function of n .

Running Time : The Worst Case

- ▶ The worst case arises when the array is in reverse sorted order
- ▶ It starts out in decreasing order.
- ▶ The running time is a quadratic function of n .

Running Time : Average Case

- In **some particular cases**, we'll be interested in the average-case running time of an algorithm.
- We'll see the technique of **probabilistic analysis** applied to various algorithms throughout this course.
- Often, we'll assume that all inputs of a given size are equally likely.
- In practice, this assumption may be violated, but we can sometimes use a randomized algorithm, which makes random choices, to allow a probabilistic analysis and yield an expected running time.

Worst-case and Average-case Analysis

- Our analysis of insertion sort looked at both the **best case**, in which the input array was already sorted, and the **worst case**, in which the input array was reverse sorted.
- We usually **concentrate on** finding only **the worst-case running time**, that is, the longest running time for any input of size n .
- **Why?**

Worst-case and Average-case Analysis

Here are three reasons:

- 1- The worst-case running time of an algorithm gives an **upper bound** on the running time for **any input**.
 - If you know it, then you have **a guarantee** that the algorithm never takes any longer.
 - This feature is especially important for **real-time computing**, in which operations must **complete by a deadline**.

Worst-case and Average-case Analysis

- 2-** For some algorithms, the worst case occurs fairly often.
- For example, in searching a database for a particular piece of information, the searching algorithm's worst case often occurs when the information is not present in the database.
 - In some applications, searches for absent information may be frequent.

Worst-case and Average-case Analysis

3- The “average case” is often roughly as bad as the worst case.

- Suppose that you run insertion sort on an array of n randomly chosen numbers.
- How long does it take to determine where in subarray $A[1 : i - 1]$ to insert element $A[i]$?
- On average, half the elements in $A[1 : i - 1]$ are less than $A[i]$, and half the elements are greater.
- On average, therefore, $A[i]$ is compared with just half of the subarray $A[1 : i - 1]$, and so t_i is about $i/2$. The resulting average-case running time turns out to be a quadratic function of the input size, just like the worst-case running time.

Units for Measuring Running Time

- We can simply use some standard unit of time measurement—a second, or millisecond, and so on—to measure the running time of a program implementing the algorithm.
- Drawbacks:
 - dependence on the speed of a particular computer,
 - dependence on the quality of a program implementing the algorithm and of the compiler used in generating the machine code,
 - the difficulty of clocking the actual running time of the program.
- We would like to have a metric that does not depend on these extraneous factors.

Units for Measuring Running Time:

Basic Operation

- One possible approach is to count the number of times each of the algorithm's operations is executed.
- This approach is both excessively difficult and usually unnecessary.
- That's why, we identify the most important operation of the algorithm,
- **Basic operation:** The operation contributing the most to the total running time
- it is usually the most time-consuming operation in the algorithm's innermost loop.

Units for Measuring Running Time:

Basic Operation

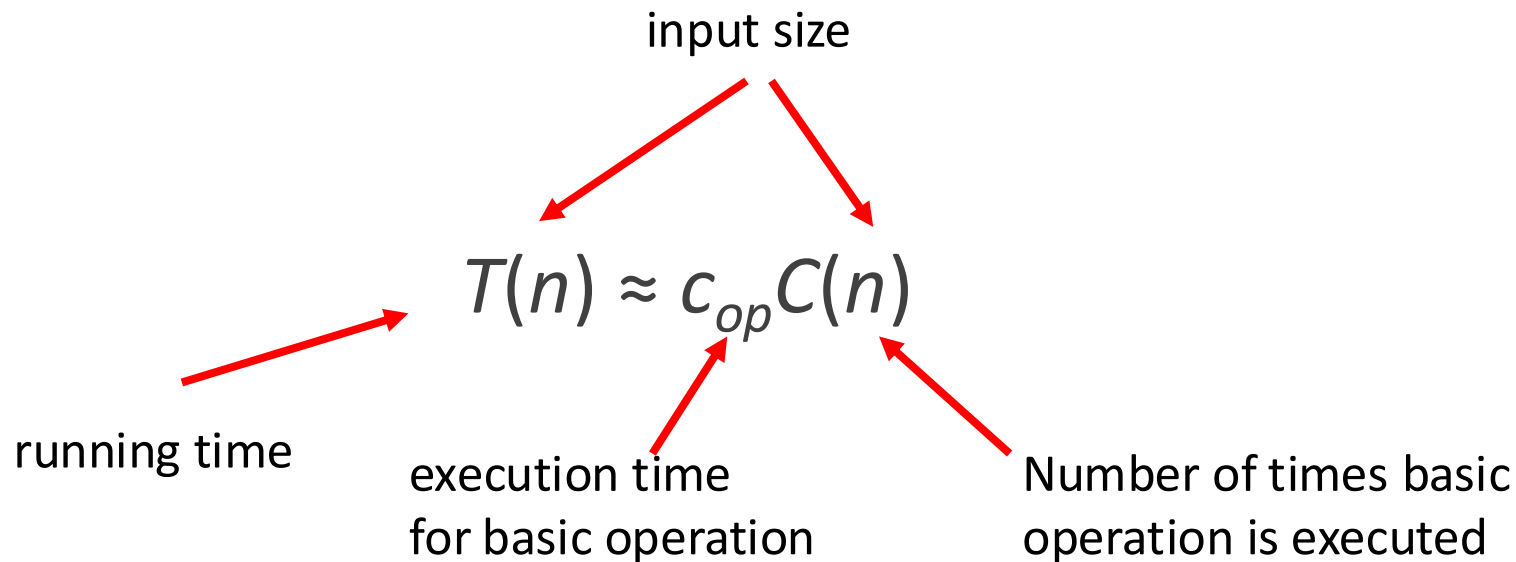
- Most **sorting algorithms** work by comparing elements (keys) of a list being sorted with each other; for such algorithms, **the basic operation is a key comparison**.
- As another example, algorithms for **mathematical problems** typically involve some or all of the **four arithmetical operations**: addition, subtraction, multiplication, and division.
- Of the four, the most time-consuming operation is **division**, followed by multiplication and then addition and subtraction, with the last two usually considered together.

Input size and basic operation examples

<i>Problem</i>	<i>Input size measure</i>	<i>Basic operation</i>
Searching for key in a list of n items	Number of list's items, i.e. n	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers
Checking primality of a given integer n	n'size = number of digits (in binary representation)	Division
Typical graph problem	#vertices and/or edges	Visiting a vertex or traversing an edge

Theoretical analysis of time efficiency

- Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size



Theoretical analysis of time efficiency

$$T(n) \approx c_{op}C(n)$$

- The formula can give a reasonable estimate of the algorithm's running time.
- Assuming that

$$C(n) = \frac{1}{2}n(n - 1)$$

- How much longer will the algorithm run if we double its input size?

Theoretical analysis of time efficiency

$$C(n) = \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2$$

and therefore

$$\frac{T(2n)}{T(n)} \approx \frac{c_{op}C(2n)}{c_{op}C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4.$$

Analysis of Insertion Sort

- For each $i = 2, 3, \dots, n$, let t_i denote the number of times the while loop test in line 5 is executed for that value of i .
- We usually denote the running time of an algorithm on an input of size n by $T(n)$.
- To compute $T(n)$, the running time of INSERTION-SORT on an input of n values, we sum the products of the cost and times columns, obtaining.

INSERTION-SORT(A, n)	cost	times
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 // Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

$$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1).$$

Analysis of Insertion Sort: The Worst Case

- We find that in the worst case, the running time of INSERTION-SORT is

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n-1).$$


$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ &\quad + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

- The running time is a quadratic function of n .

Order of Growth

- We consider only the **leading term of a formula** (e.g., an^2), since the lower-order terms are relatively insignificant for large values of n .
- We also ignore the leading term's constant coefficient, since constant factors are less significant than the rate of growth in determining computational efficiency for large inputs.
- For insertion sort's worst-case running time, when we ignore the lower-order terms and the leading term's constant coefficient, only the factor of n^2 from the leading term remains.
- That factor, n^2 , is by far the **most important part** of the running time.

Order of Growth

- For example, suppose that an algorithm implemented on a particular machine takes $n^2/100 + 100n + 17$ microseconds on an input of size n .
- Although the coefficients of $1/100$ for the n^2 term and 100 for the n term differ by four orders of magnitude, the $n^2/100$ term dominates the $100n$ term once n exceeds $10,000$.
- Although $10,000$ might seem large, it is smaller than the population of an average town.
- Many real-world problems have much larger input sizes.

Order of Growth

- To highlight the **order of growth of the running time**, we have a special notation that uses the Greek letter Θ (**theta**).
- We write that insertion sort has a **worst-case** running time of $\Theta(n^2)$
- We also write that insertion sort has a **best-case** running time of $\Theta(n)$.
- For now, think of Θ -notation as saying “roughly proportional when n is large,”
- So that $\Theta(n^2)$ means “roughly proportional to n^2 when n is large”
- $\Theta(n)$ means “roughly proportional to n when n is large.”

Order of Growth

- We usually consider one algorithm to be **more efficient** than another if its **worst-case running time** has a **lower order** of growth.
- Due to constant factors and lower-order terms, an algorithm whose running time has a higher order of growth might take **less time for small inputs** than an algorithm whose **running time has a lower order of growth**.
- But on large enough inputs, an algorithm whose worst-case running time is $\Theta(n^2)$, for example, takes less time in the worst case than an algorithm whose worst-case running time is $\Theta(n^3)$.
- Regardless of the constants hidden by the Θ -notation, there is always some number, say n_0 , such that for all input sizes $n \geq n_0$, the $\Theta(n^2)$ algorithm beats the $\Theta(n^3)$ algorithm in the worst case.

Order of growth

- Example: $c n^2$
 - How much faster will algorithm run on computer that is twice as fast?
 - How much longer does it take to solve problem of double input size?

Order of growth

- Example: $c n^2$
 - How much faster will algorithm run on computer that is twice as fast?
 - Answer: 2
 - How much longer does it take to solve problem of double input size?
 - Answer: 4

Values of some important functions as $n \rightarrow \infty$

- The function growing the **slowest** among these is the **logarithmic function**.
- On the other end of the spectrum are the exponential function 2^n and the factorial function $n!$
- Both **these functions grow so fast** that their values become astronomically large even for rather small values of n .

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Values of some important functions as $n \rightarrow \infty$

- For example, it would take about $4 \cdot 10^{10}$ years for a computer making a trillion (10^{12}) operations per second to execute 2^{100} operations.
- Though this is incomparably faster than it would have taken to execute $100!$ operations, it is still longer than 4.5 billion ($4.5 \cdot 10^9$) years— the estimated age of the planet Earth.
- There is a tremendous difference between the orders of growth of the functions 2^n and $n!$.
- Algorithms that require an exponential number of operations are practical for solving only problems of **very small sizes**.

Values of some important functions as $n \rightarrow \infty$

- Another way to appreciate the qualitative difference among the orders of growth of the functions in table is to consider how they react to, say, a **twofold** increase in the value of their argument n .

- The function $\log_2 n$ increases in value by just 1

$$\log_2 2n = \log_2 2 + \log_2 n = 1 + \log_2 n$$

- The linear function increases twofold, the linearithmic function $n \log_2 n$ increases slightly more than twofold; the quadratic function n^2 and cubic function n^3 increase fourfold and eightfold, respectively.

$$(2n)^2 = 4n^2 \text{ and } (2n)^3 = 8n^3$$

- The value of 2^n gets squared

$$\text{because } 2^{2n} = (2^n)^2$$

- $n!$ increases much more than that

Example: Sequential search

- It is reasonable to measure an algorithm's efficiency as a function of a parameter indicating the **size of the algorithm's input**.
- There are many algorithms for which running time depends not only on an input size but also on **the specifics of a particular input**.
- **Sequential search**: This is a straightforward algorithm that searches for a given item (some search key K) in a list of n elements by checking successive elements of the list until either a match with the search key is found or the **list is exhausted**.

Example: Sequential search

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element in A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

Best-case, average-case, worst-case

- For some algorithms efficiency depends on form of input:
- Worst case: $C_{\text{worst}}(n)$ – maximum over inputs of size n
- Best case: $C_{\text{best}}(n)$ – minimum over inputs of size n
- Average case: $C_{\text{avg}}(n)$ – “average” over inputs of size n
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations considered as a random variable under some assumption about the probability distribution of all possible inputs

Best-case, average-case, worst-case

- Worst case: When there are **no matching elements** or the first matching element happens **to be the last one** on the list, the algorithm makes the largest number of key comparisons among all possible inputs of size n

$$C_{worst}(n) = n$$

- It **guarantees** that for any instance of size n , the running time will not exceed $C_{worst}(n)$.

Best-case, average-case, worst-case

- **Best case:** An input (or inputs) of size n for which the algorithm runs the fastest among all possible inputs of that size.
- The best-case inputs for sequential search are lists of size n with their first element equal to a search key.

$$C_{\text{best}}(n) = 1$$

- The analysis of the best-case efficiency is not nearly as important as that of the worst-case efficiency.

Best-case, average-case, worst-case

- **Average case** : It is clear that neither the worst-case analysis nor its best-case counterpart yields the necessary information about an algorithm's behavior on a “typical” or “random” input.
- Let's consider again sequential search. The standard assumptions are that
 - The probability of a successful search is equal to p ($0 \leq p \leq 1$)
 - The probability of the first match occurring in the i^{th} position of the list is the same for every i .

Best-case, average-case, worst-case

- Under these assumptions, we can find the average number of key comparisons

$$\begin{aligned}C_{avg}(n) &= \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + i \cdot \frac{p}{n} + \dots + n \cdot \frac{p}{n}\right] + n \cdot (1 - p) \\&= \frac{p}{n} [1 + 2 + \dots + i + \dots + n] + n(1 - p) \\&= \frac{p}{n} \frac{n(n+1)}{2} + n(1 - p) = \frac{p(n+1)}{2} + n(1 - p).\end{aligned}$$

- if $p = 1$ (the search must be successful), the average number of key comparisons made by sequential search is $(n + 1)/2$; that is, the algorithm will inspect, on average, about half of the list's elements.
- If $p = 0$ (the search must be unsuccessful), the average number of key comparisons will be n because the algorithm will inspect all n elements on all such inputs.

Main Points

- Both time and space efficiencies are measured as functions of the algorithm's input size.
- **Time efficiency** is measured by counting the number of times the algorithm's **basic operation** is executed.
- **Space efficiency** is measured by counting the number of extra memory units consumed by the algorithm.
- The efficiencies of some algorithms may differ significantly for inputs of the same size.
- For such algorithms, we need to distinguish between the **worst-case**, **average-case**, and **best-case** efficiencies.

Summary

- Analysis of Algorithms
- Euclid's Algorithm
- Sieve Algorithm
- Sequential Search
- Running Time: The worst, best, and average case
- Order of growth comparison

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.